

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#02)

Implementation of Combinational Circuits using Verilog Gate & Dataflow Level Modeling

Student Name: Muhammad Ashnab Safdar Roll No: I24-6500 Section: CE-A

Date performed: 25/8/2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:

In this task we have to make Half Adder using XOR and AND gates and then using Half Adder make a Full Adder using DATA FLOW Level and Gate Level, verify it using a testbench in modelsim and then also by making same circuit in Logisim

Code:

```
module fullAdder(Sum,Cout,A,B,C);

wire s1,c1,c2;
output Sum,Cout;
input A,B,C;

halfAdder ha1(s1,c1,A,B);
halfAdder ha2(Sum,c2,s1,C);
or (Cout,c1,c2);

endmodule

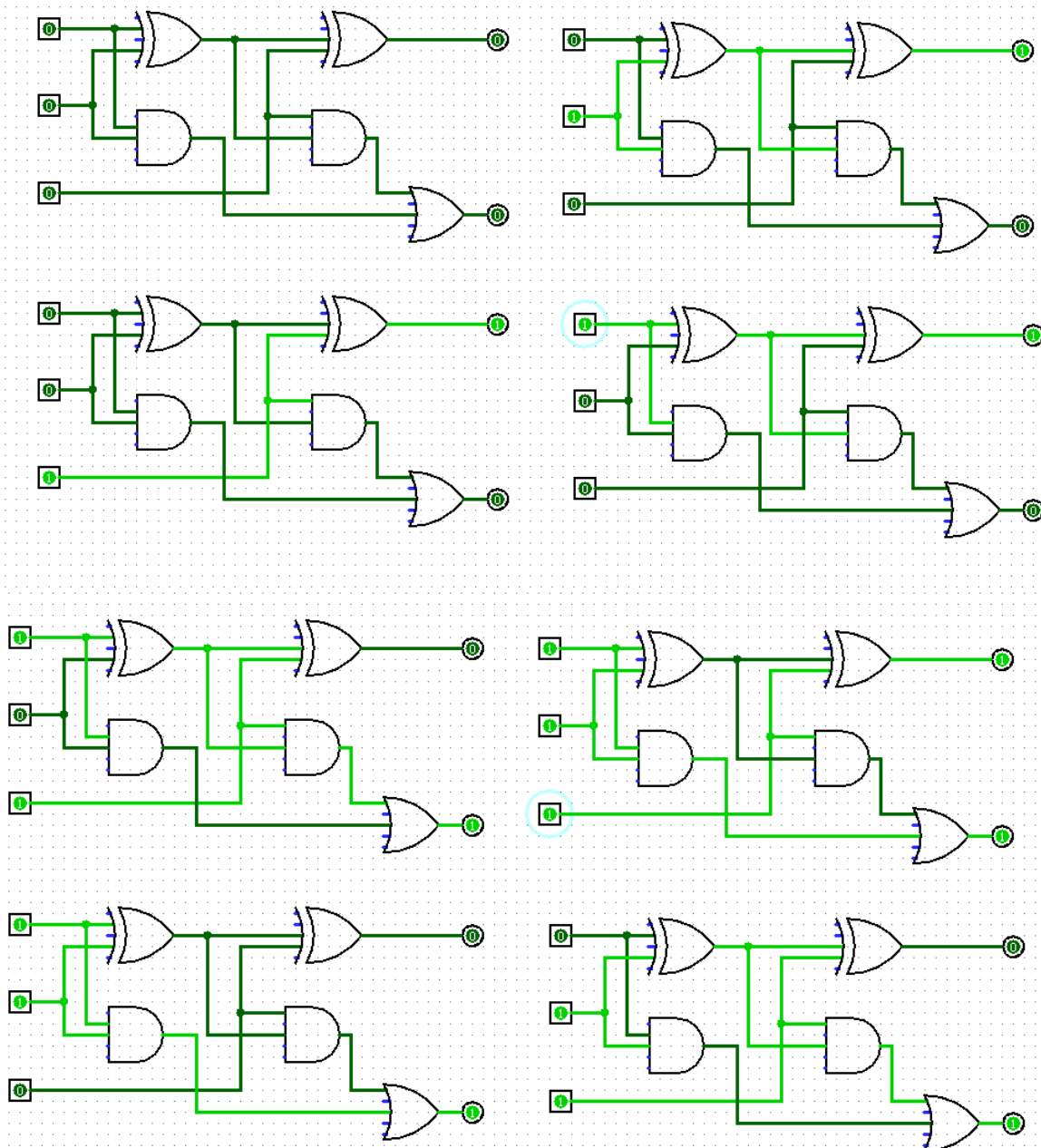
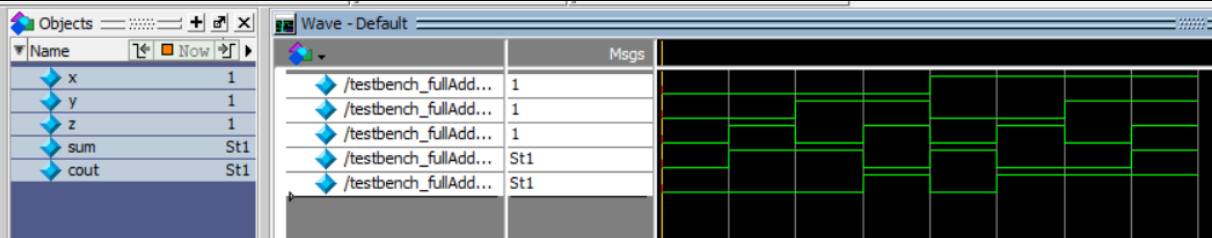
module halfAdder(sum1,cout1,a,b);

input a,b;
output sum1,cout1;
//FOR GATE LEVEL: (use any one at a time)
xor(sum1,a,b);
and(cout1,a,b);
//FOR DATA FLOW: (use any one at a time)
assign sum1 =a^b;
assign cout1= a&b;

endmodule

module testbench_fullAdder ();
reg x,y,z;
wire sum,cout;
fullAdder uut(.Sum(sum), .Cout(cout), .A(x), .B(y), .C(z));
initial
begin
    x = 0; y = 0; z = 0; #50; // 000
    x = 0; y = 0; z = 1; #50; // 001
    x = 0; y = 1; z = 0; #50; // 010
    x = 0; y = 1; z = 1; #50; // 011
    x = 1; y = 0; z = 0; #50; // 100
    x = 1; y = 0; z = 1; #50; // 101
    x = 1; y = 1; z = 0; #50; // 110
    x = 1; y = 1; z = 1; #50; // 111
end
endmodule
```

Output:



Task 2:

Description:

// Write description of the task here  (max. five to seven lines)

Code:

```
module _3x8Decoder(x,y,z,a,b,c,d,e,f,g,h);  
input x,y,z;  
output a,b,c,d,e,f,g,h;  
wire xn,yn,zn;
```

FOR GATE FLOW LEVEL: (use any one at a time)

```
not (xn, x);  
not (yn, y);  
not (zn, z);  
  
and (a, xn, yn, zn); // 000  
and (b, xn, yn, z); // 001  
and (c, xn, y, zn); // 010  
and (d, xn, y, z); // 011  
and (e, x, yn, zn); // 100  
and (f, x, yn, z); // 101  
and (g, x, y, zn); // 110  
and (h, x, y, z); // 111
```

FOR DATA FLOW LEVEL: (use any one at a time)

```
assign xn=~x;  
assign yn=~y;  
assign zn=~z;  
  
assign a = xn & yn & zn; // 000  
assign b = xn & yn & z; // 001  
assign c = xn & y & zn; // 010  
assign d = xn & y & z; // 011  
assign e = x & yn & zn; // 100  
assign f = x & yn & z; // 101  
assign g = x & y & zn; // 110  
assign h = x & y & z; // 111
```

```
endmodule  
module testbench_decoder3x8();  
reg x, y, z;  
wire a, b, c, d, e, f, g, h;
```

```
_3x8Decoder uut(.x(x), .y(y), .z(z), .a(a), .b(b), .c(c), .d(d), .e(e), .f(f), .g(g), .h(h));
```

```
initial begin
```

```

x=0; y=0; z=0; #50; // 000 -> a
x=0; y=0; z=1; #50; // 001 -> b
x=0; y=1; z=0; #50; // 010 -> c
x=0; y=1; z=1; #50; // 011 -> d
x=1; y=0; z=0; #50; // 100 -> e
x=1; y=0; z=1; #50; // 101 -> f
x=1; y=1; z=0; #50; // 110 -> g
x=1; y=1; z=1; #50; // 111 -> h

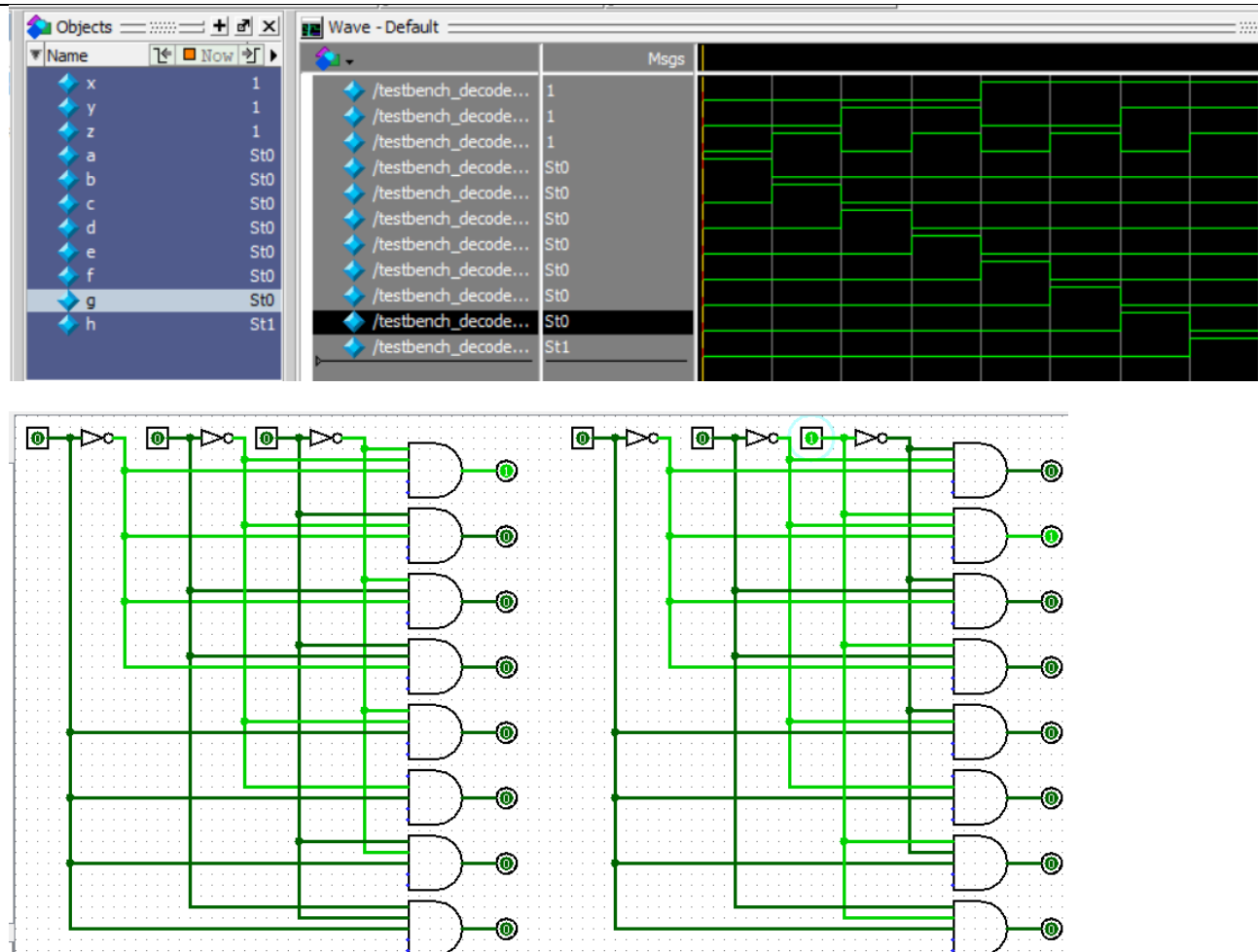
```

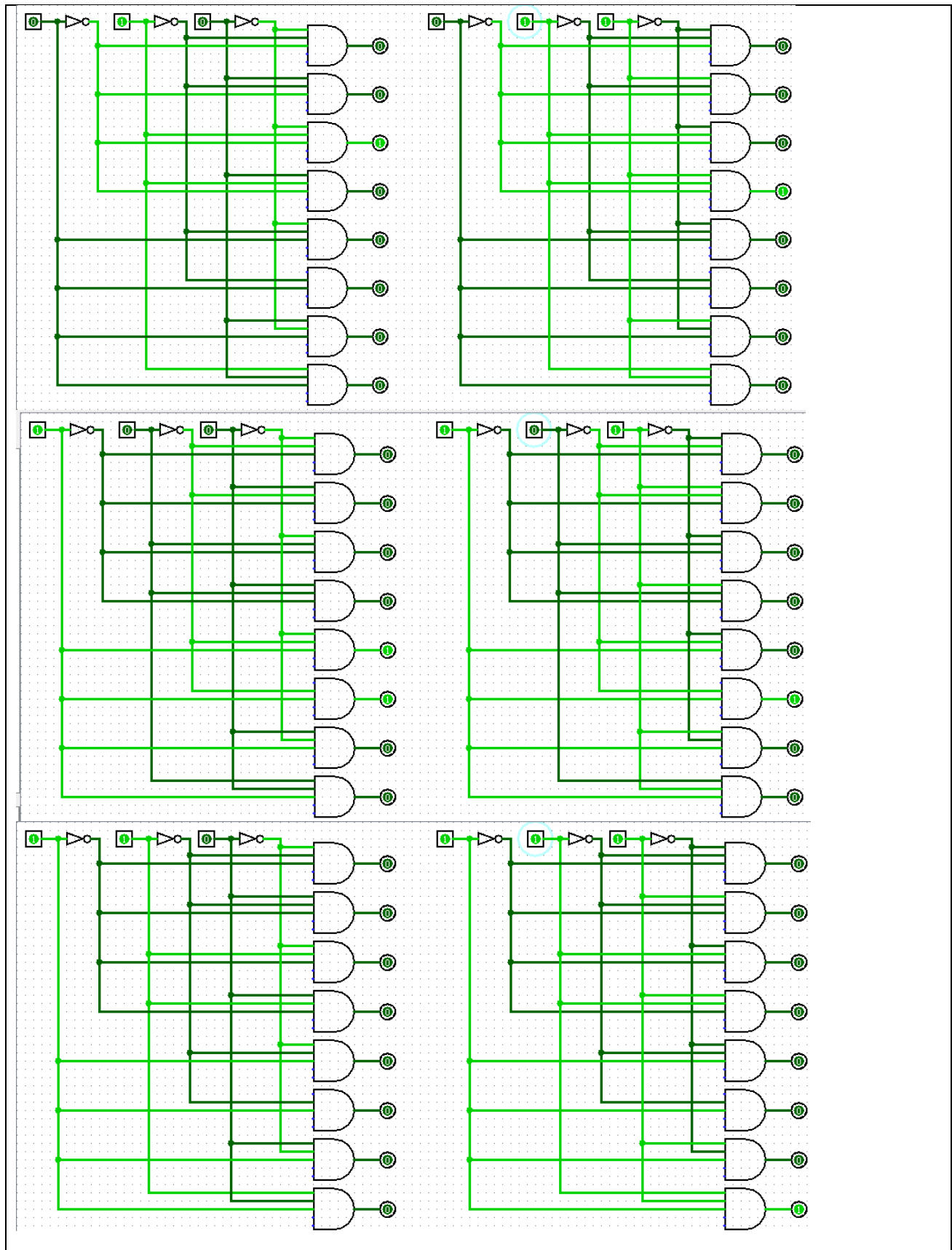
```

end
endmodule

```

Output:





Task 3:

Description:

In this task we have to make 8x1 Multiplexer using OR and AND gates and then using DATA FLOW Level and Gate Level, verify it using a testbench in modelsim and then also by making same circuit in Logisim

Code:

```
module _8x1Mux(x,y,z,a,b,c,d,e,f,g,h,out);  
input x,y,z,a,b,c,d,e,f,g,h;  
output out;  
wire xn,yn,zn,a1,b1,c1,d1,e1,f1,g1,h1;
```

USING GATE FLOW LEVEL: (use any one at a time)

```
not (xn, x);  
  not (yn, y);  
  not (zn, z);  
  
  and (a1, xn, yn, zn, a); // 000  
  and (b1, xn, yn, z, b); // 001  
  and (c1, xn, y, zn, c); // 010  
  and (d1, xn, y, z, d); // 011  
  and (e1, x, yn, zn, e); // 100  
  and (f1, x, yn, z, f); // 101  
  and (g1, x, y, zn, g); // 110  
  and (h1, x, y, z, h); // 111
```

```
  or (out, a1, b1, c1, d1, e1, f1, g1, h1);
```

USING DATA FLOW LEVEL: (use any one at a time)

```
assign xn=~x;  
assign yn=~y;  
assign zn=~z;  
  
  assign a1 = xn & yn & zn &a; // 000  
  assign b1 = xn & yn & z &b; // 001  
  assign c1 = xn & y & zn&c; // 010  
  assign d1 = xn & y & z&d; // 011  
  assign e1 = x & yn & zn&e; // 100  
  assign f1 = x & yn & z&f; // 101  
  assign g1 = x & y & zn&g; // 110  
  assign h1 = x & y & z&h; // 111  
  
assign out=a1|b1|c1|d1|e1|f1|g1|h1;  
  
endmodule  
module testbench_mux8x1();  
  reg x, y, z;  
  reg a, b, c, d, e, f, g, h;  
  wire out;
```



```
_8x1Mux uut(.x(x), .y(y), .z(z),.a(a), .b(b), .c(c), .d(d),.e(e), .f(f), .g(g), .h(h),.out(out));
```

```
initial begin
```

```
    a = 0; b = 1; c = 0; d = 1;  
    e = 0; f = 1; g = 0; h = 1;
```

```
    x=0; y=0; z=0; #50; // 000 -> out should = a = 0  
    x=0; y=0; z=1; #50; // 001 -> out should = b = 1  
    x=0; y=1; z=0; #50; // 010 -> out should = c = 0  
    x=0; y=1; z=1; #50; // 011 -> out should = d = 1  
    x=1; y=0; z=0; #50; // 100 -> out should = e = 0  
    x=1; y=0; z=1; #50; // 101 -> out should = f = 1  
    x=1; y=1; z=0; #50; // 110 -> out should = g = 0  
    x=1; y=1; z=1; #50; // 111 -> out should = h = 1
```

```
end
```

```
initial begin
```

```
    a = 0; b = 1; c = 0; d = 1;  
    e = 0; f = 1; g = 0; h = 1;
```

```
    x=0; y=0; z=0; #50; // 000 -> out should = a = 0  
    x=0; y=0; z=1; #50; // 001 -> out should = b = 1  
    x=0; y=1; z=0; #50; // 010 -> out should = c = 0  
    x=0; y=1; z=1; #50; // 011 -> out should = d = 1  
    x=1; y=0; z=0; #50; // 100 -> out should = e = 0  
    x=1; y=0; z=1; #50; // 101 -> out should = f = 1  
    x=1; y=1; z=0; #50; // 110 -> out should = g = 0  
    x=1; y=1; z=1; #50; // 111 -> out should = h = 1
```

```
end
```

```
endmodule
```

Output:

